Phase 2 Implementation Report

Operating Systems Fall 2025

Khaleeqa Aasiyah Garrett

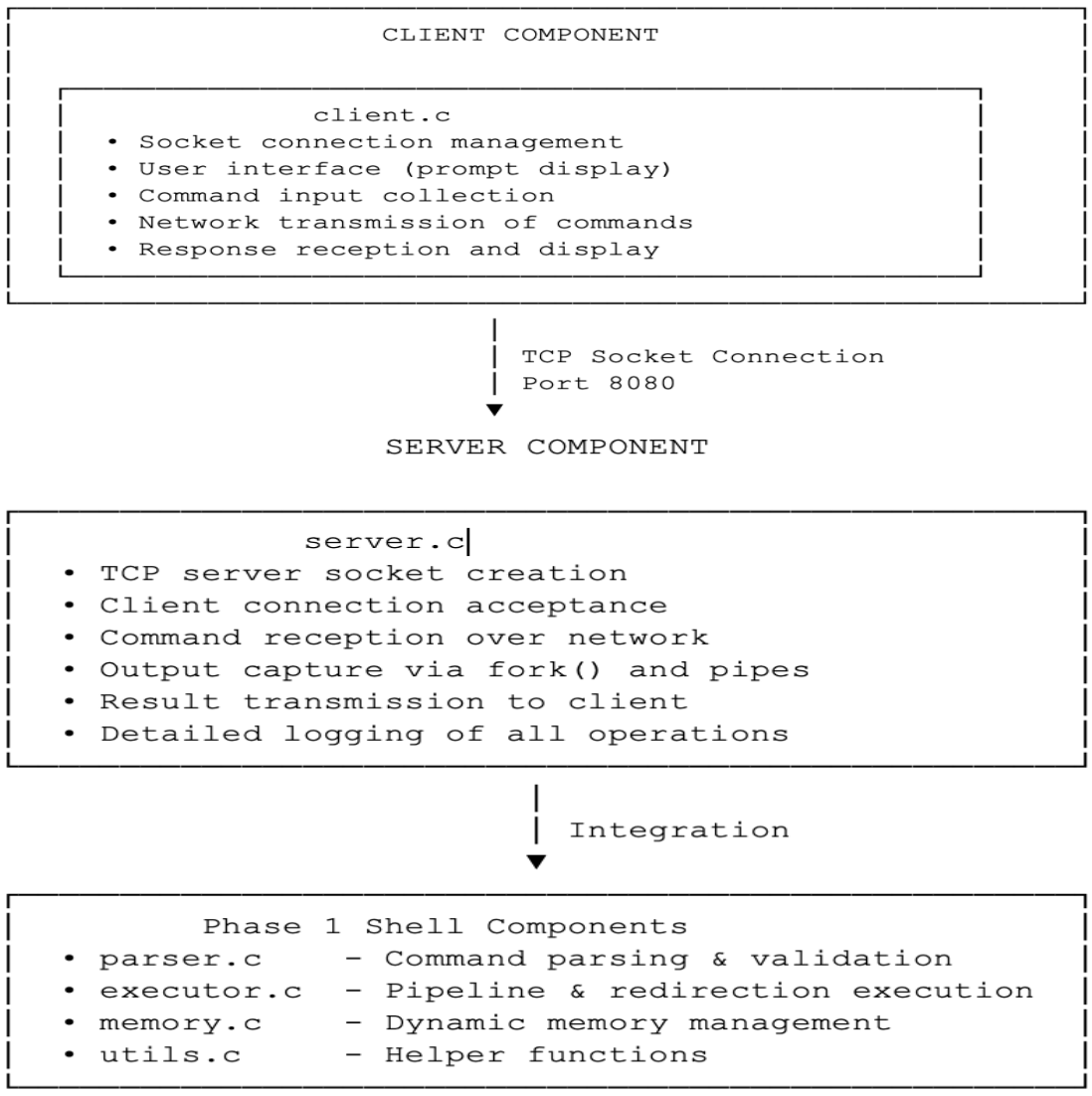
Kag9691 | Custom Shell Implementation

Architecture and Design

High-Level System Architecture

Phase 2 extends MyShell from Phase 1 by implementing a client-server architecture that enables remote command execution over TCP/IP networks. The system maintains all Phase 1 functionality while adding network communication capabilities through socket programming.

My Network Shell Architecture looks like this:



1. Server (server executable): A TCP server that listens on **PORT 8080**. It is responsible for:
    - Accepting a single, sequential client connection.
    - Receiving raw command strings from the client.
    - Parsing the command using the Phase 1 core logic.
    - Executing the command non-interactively while capturing all output.
    - Sending the captured output (stdout + stderr) back to the client

2. Client (client executable): A command-line application that acts as the user's interface. It is responsible for:
    - Establishing a connection to the server.
    - Presenting the shell prompt (`$ `) and reading user input.
    - Transmitting the raw input string to the server.
    - Receiving and displaying the server's response.

3. Shell Core (Phase 1 Objects): The existing modular components (`executor.o`, `parser.o`, `memory.o`, `utils.o`) are linked directly into the `server` executable. This ensures all existing functionality (pipelines, redirection, command validation) is retained and executed remotely on the server.

Key Design Decisions

- **Re-use of Modular Core:** By linking the Phase 1 core directly to `server.c`, we achieved rapid integration and maintained the robustness of the command execution logic, focusing Phase 2 efforts entirely on networking and I/O capture.
- **Dedicated I/O Capture (Server):** The function `capture_command_output` is central to the server's operation. It uses the `fork()` system call combined with **two separate `pipe()` calls** (one for stdout, one for stderr) to redirect the command's output streams into memory buffers. This is a crucial design to prevent the command's output from being lost to the server's console and ensures both normal output and errors are captured for the client.
- **Sequential Server Model:** The server is implemented sequentially, handling one client connection at a time. This simplifies resource management and avoids complex issues related to concurrent process handling in this phase

File Structure and Organization

The Phase 2 implementation follows the modular structure established in Phase 1, with two new files dedicated to networking:

File	Role	Dependencies
`server.c`	Network listener, client handler, **I/O capture logic**	`shell.h`, Phase 1 core objects
`client.c`	User interface, network connection, send/receive logic	Standard C libraries, `netinet/in.h`
`executor.c`, `parser.c`, etc.	Phase 1 core logic (Execution, Parsing, etc.)	`shell.h`
`shell.h`	Common headers, constants, and `Command`/`Pipeline`structs	Phase 1/Phase 2 files
`Makefile`	Compilation script for `server` and `client` executables	All source files

Implementation Highlights

1. Server I/O Capture (`server.c`)

The most critical function in Phase 2 is `capture_command_output`, which bridges the Phase 1 execution logic with the network output stream.

```
// server.c - Simplified capture_command_output logic
int stdout_pipe[2];
int stderr_pipe[2];

if (pipe(stdout_pipe) == -1 || pipe(stderr_pipe) == -1) { /* error */ }

pid_t pid = fork();

if (pid == 0) { // CHILD PROCESS
    close(stdout_pipe[0]); // Close read ends
    close(stderr_pipe[0]);

    // Key operation: Redirect STDOUT/STDERR to pipe write ends
    dup2(stdout_pipe[1], STDOUT_FILENO);
    dup2(stderr_pipe[1], STDERR_FILENO);

    // Execute the command using the Phase 1 executor
    execute_pipeline(pipeline); // Execution function from executor.c
    exit(EXIT_SUCCESS);
} else { // PARENT PROCESS
    close(stdout_pipe[1]); // Close write ends
    close(stderr_pipe[1]);

    // Read from the read ends of the pipes into buffers
    read(stdout_pipe[0], output_buf, output_size - 1);
    read(stderr_pipe[0], error_buf, error_size - 1);

    // Wait for child to finish and cleanup
    waitpid(pid, NULL, 0);
    close(stdout_pipe[0]);
    close(stderr_pipe[0]);
}
```

This function ensures that the parent process can read all output generated by the child process (which runs the user command) before packaging the combined output (stdout + stderr) into a single response buffer for the client.

2. Network Communication and Error Handling

Server Edge Case Handling (Empty Output): A successful command (e.g., `touch newfile.txt` or a command that produces no output) would result in a zero-length network response, which could be misinterpreted as a connection failure by the client.

- **Resolution:** The server defines an `EMPTY_RESPONSE_MARKER` as a single byte (`\x01`). If a command executes successfully but generates no output, the server sends only this marker.

Client Edge Case Handling (Response Interpretation): The client must differentiate between a true response and the empty marker.

- **Resolution:** The client checks the received data: `if (bytes_received == 1 && response_buffer[0] == '\x01')`. If this condition is met, the client simply continues to the next prompt without printing anything, correctly handling the "no output" case. The client also adds a trailing newline if the server response doesn't already have one, ensuring clean output formatting.

3. Client REPL Loop (`client.c`)

The client operates in a continuous request-response loop:

1. **Prompt & Read:** Calls `display_prompt()` and `read_input()`.
2. **Send:** Transmits the user command to the server via `send()`.
3. **Receive:** Blocks, waiting for the full output from the server via `recv()`.
4. **Display:** Processes the response (checking for the marker) and prints the result.
5. **Termination:** The loop exits gracefully if the user types `exit` or if the server disconnects.

Execution Instructions

The project is compiled using the provided `Makefile` and executed by running the two separate executables.

1. Compilation

1. Open a terminal in the project directory.
2. Run the make command to build both the server and client:

Bash

```
make all
```

This creates two executables: `./server` and `./client`.

2. Running the Server

1. Open a terminal to start the server:

   Bash

   ```
   ./server
   ```

   The server will initialize and begin listening for client connections on PORT 8080.

3. Running the Client

1. Open a different terminal and return to the project directory and run the client.

   Bash

   ```
   ./client
   ```

2. The client will display the prompt (`$ `) and is ready to accept commands.

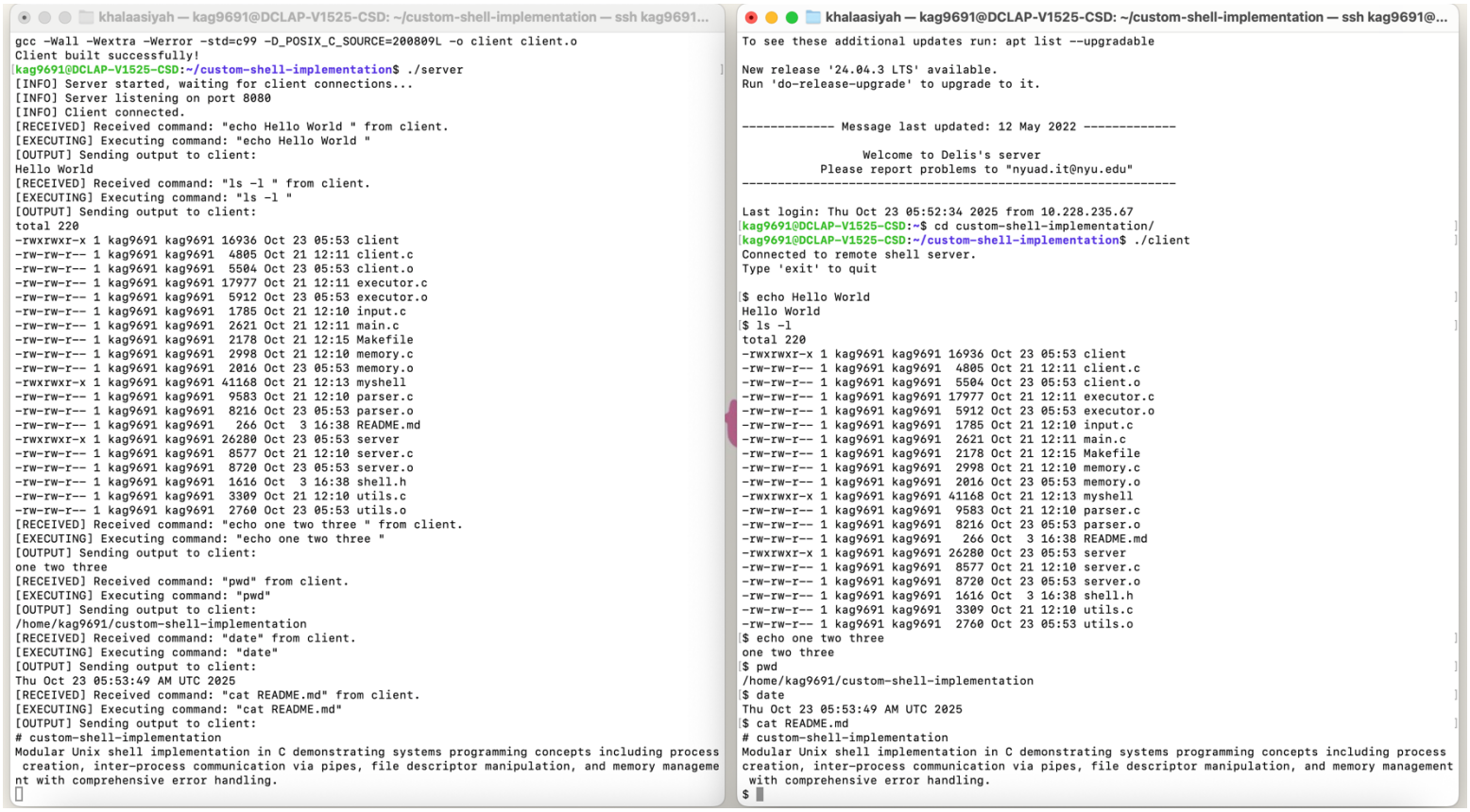
Testing

1. Basic Commands

Test Commands:

```
$ echo Hello World
$ ls -l
$ echo one two three
$ pwd
$ date
$ cat README.md
```

Screenshot 1: Basic Commands



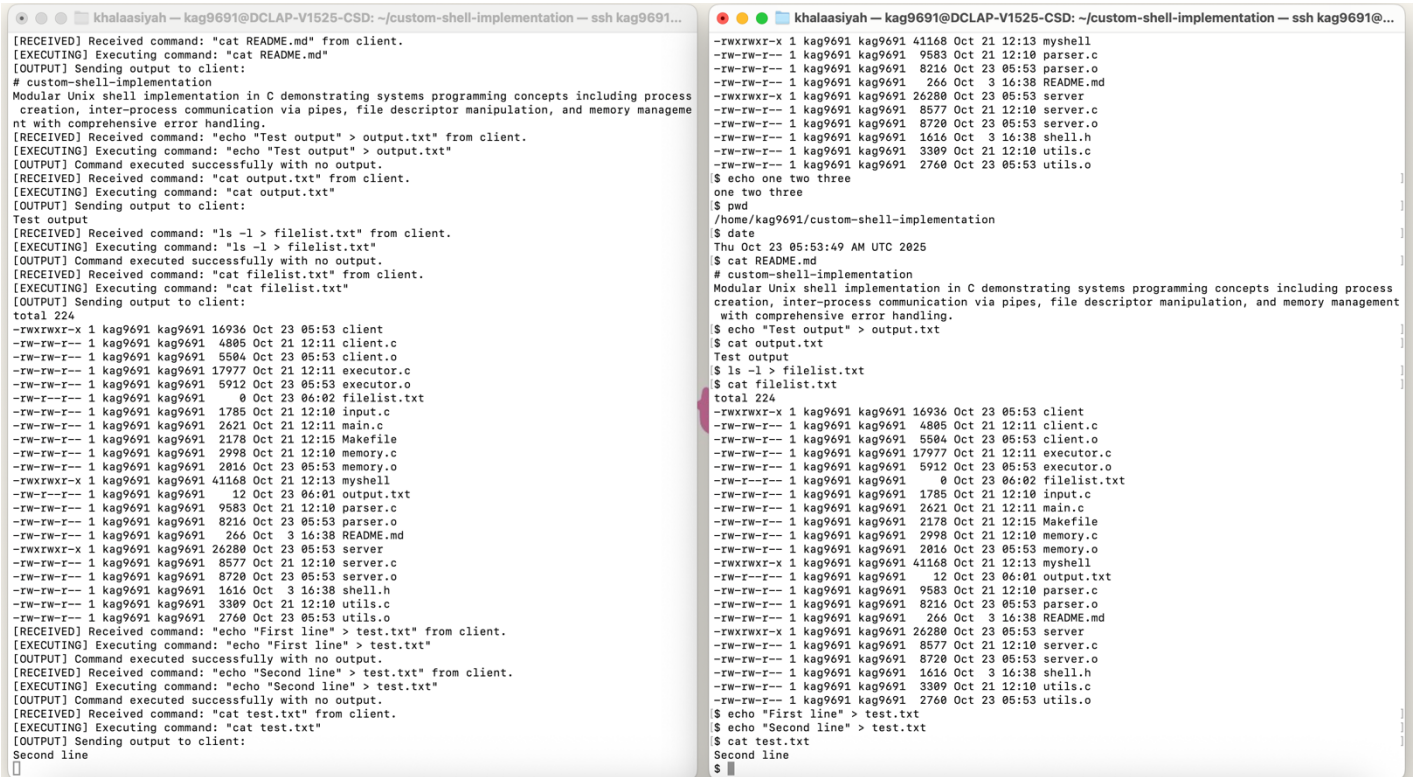
Explanation: These tests verify that the shell can execute simple external commands and properly pass arguments to them. The echo command tests basic execution and multiple argument passing. Commands like ls -l validate that flags are correctly parsed and passed. The pwd, date, and cat commands test that the shell can execute standard Unix utilities and capture their output. These tests ensure the core functionality works: the server receives commands from the client, parses them into tokens, executes them using execvp(), captures stdout/stderr through pipes, and sends the results back to the client over the socket connection.

2. Output Redirection (>)

Test Commands:

```
$ echo "Test output" > output.txt

$ cat output.txt

$ ls -l > filelist.txt

$ cat filelist.txt

$ echo "First line" > test.txt

$ echo "Second line" > test.txt

$ cat test.txt
```

Screenshot 2: Output Redirection (>)



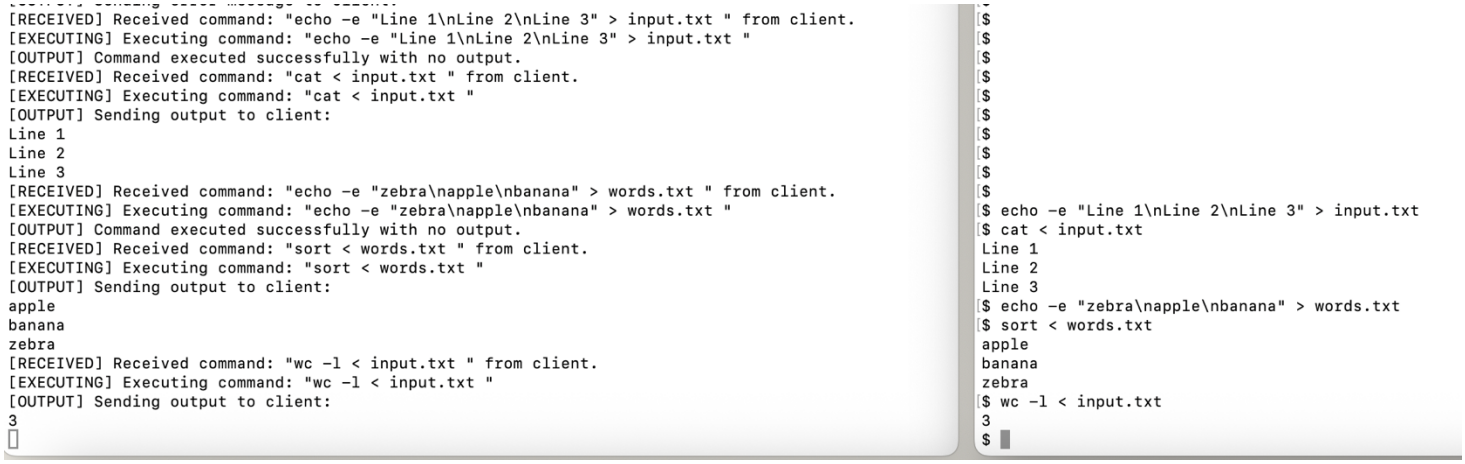
Explanation: These tests validate the output redirection feature, which redirects a command's stdout to a file instead of displaying it on the terminal. The parser must correctly identify the > operator and separate it from the filename. The executor must open the output file with O_WRONLY | O_CREAT | O_TRUNC flags using open(), then use dup2() to redirect STDOUT_FILENO to the file descriptor. Commands with output redirection should produce no visible output to the client (empty response marker is sent). The file overwriting test confirms that the O_TRUNC flag is properly used, causing subsequent redirections to the same file to erase previous content. These tests are critical because output redirection is one of the most commonly used shell features in scripting and automation.

3. Input Redirection (<)

Test Commands:

```
$ echo -e "Line 1\nLine 2\nLine 3" > input.txt

$ cat < input.txt

$ echo -e "zebra\napple\nbanana" > words.txt

$ sort < words.txt

$ wc -l < input.txt
```

Screenshot 3: Input Redirection (<)



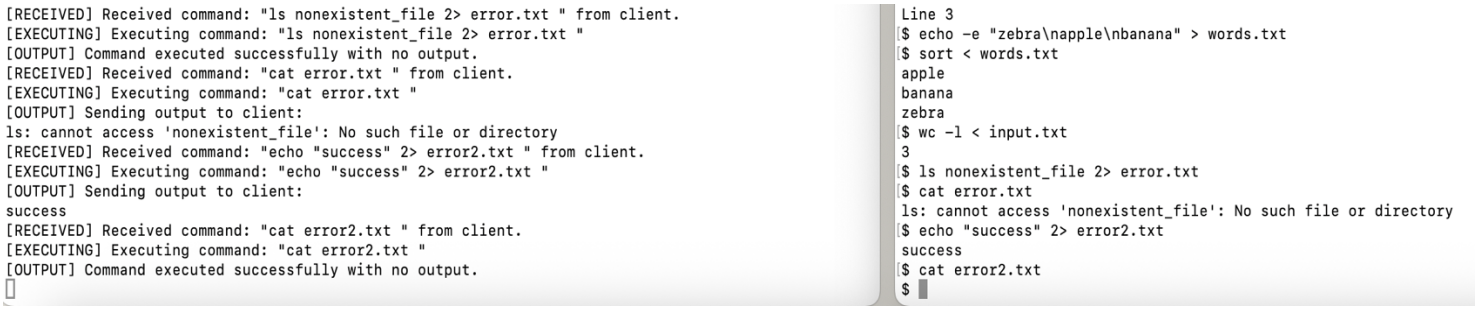
Explanation: Input redirection tests validate that commands can read data from files instead of stdin. The parser identifies the <operator and extracts the input filename. During validation, the shell checks that the input file exists and is readable using access(). The executor opens the input file with O_RDONLY and uses dup2() to redirect STDIN_FILENO to read from the file. This allows commands that normally read from the keyboard (like cat, sort, wc) to instead process file contents. The data flow is: file → stdin → command → stdout → client. These tests ensure proper file descriptor manipulation and validate that commands receive the file data as if it were typed by the user.

4. Error Redirection (2>)

Test Commands:

```
$ ls nonexistent_file 2> error.txt
$ cat error.txt
$ echo "success" 2> error2.txt
$ cat error2.txt
```

Screenshot 4: Error Redirection (2>)



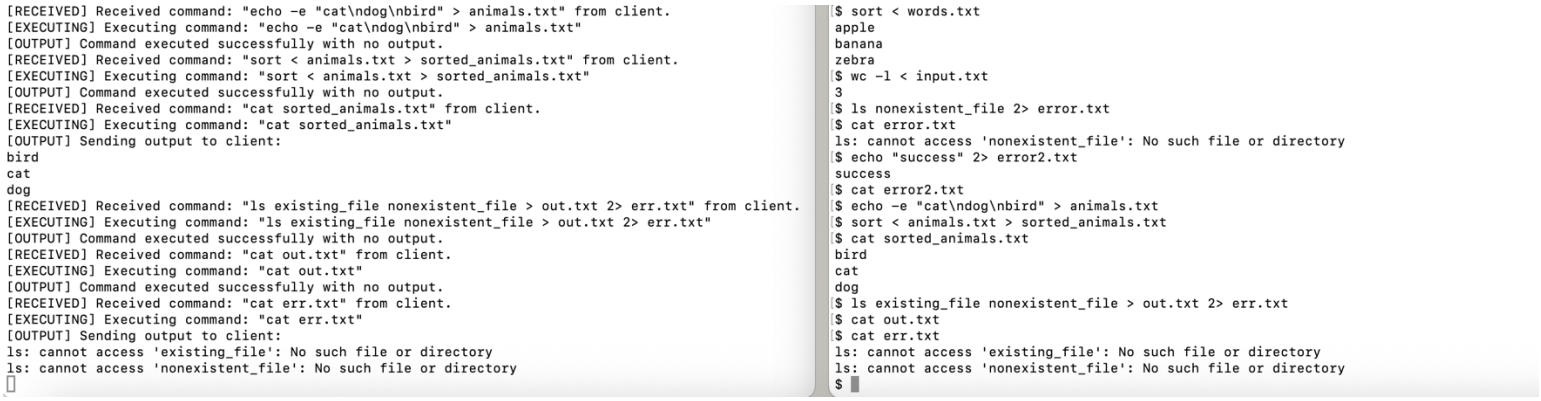
Explanation:

Error redirection tests verify that stderr (file descriptor 2) can be redirected independently from stdout. The parser must recognize 2> as a special operator distinct from regular output redirection. The executor redirects STDERR_FILENO to the specified file using dup2(). When a command fails or produces diagnostic output, those messages go to stderr instead of stdout. By redirecting stderr to a file, error messages can be captured separately from normal output. The second test confirms that when a command succeeds with no errors, the error file remains empty, proving that stdout and stderr are truly independent streams. This is essential for debugging scripts and logging, allowing users to separate successful results from error conditions.

5. Combined Redirections

```
$ echo -e "cat\ndog\nbird" > animals.txt

$ sort < animals.txt > sorted_animals.txt

$ cat sorted_animals.txt

$ ls existing_file nonexistent_file > out.txt 2> err.txt

$ cat out.txt

$ cat err.txt
```

Screenshot 5: Combined Redirections



Explanation:

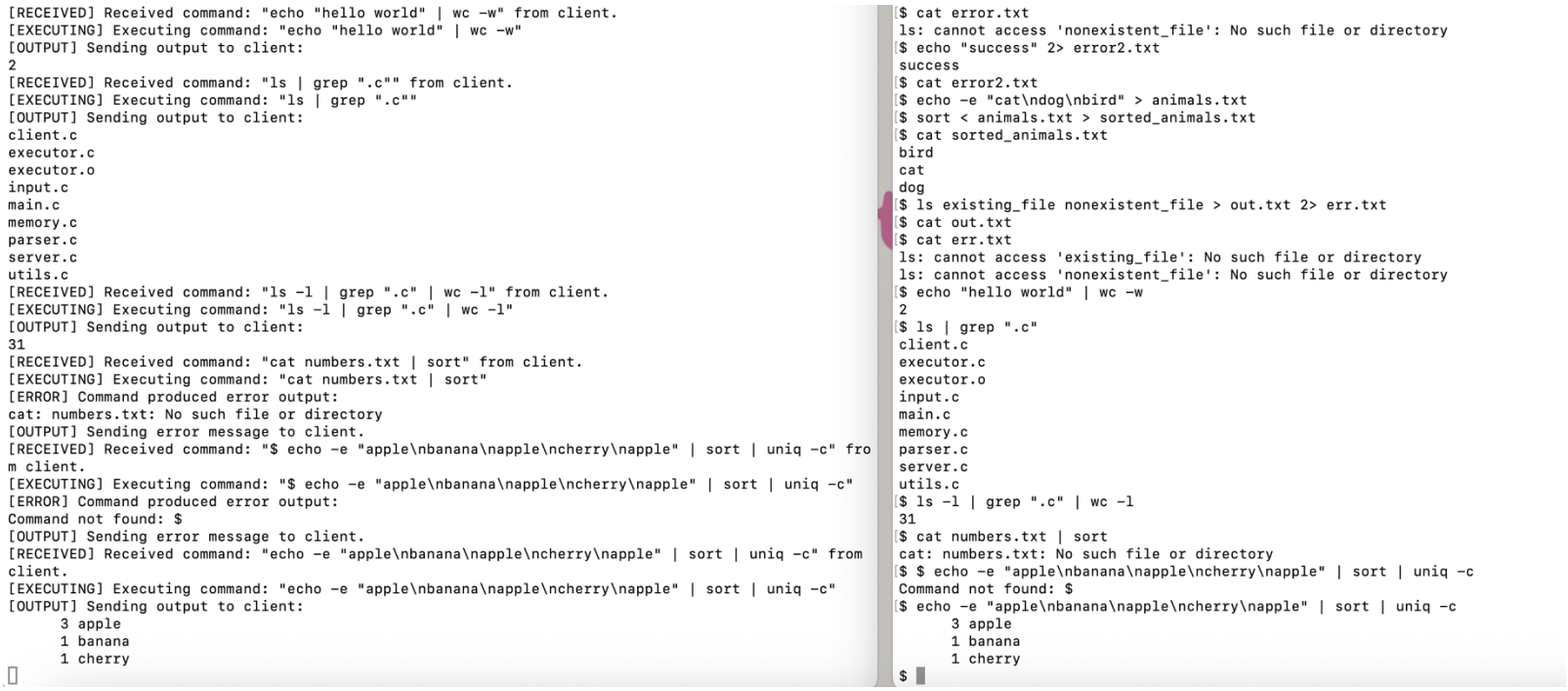
Combined redirection tests validate that multiple I/O redirections can coexist in a single command without conflicts. The first test combines input and output redirection: data flows from the input file to the command and then to the output file. The second test uses both output and error redirection simultaneously, demonstrating that stdout and stderr can go to different destinations. The executor must handle multiple file openings and multiple dup2() calls in the correct order. These tests ensure the parser can identify multiple redirection operators in one command, the Command structure can store multiple file paths, and the executor properly sets up all redirections before executing the command. This functionality is crucial for production scripts that need to log normal results and errors separately.

6. Pipes (|)

Test Command:

```
$ echo "hello world" | wc -w

$ ls | grep ".c"

$ ls -l | grep ".c" | wc -l

$ echo -e "apple\nbanana\napple\ncherry\napple" | sort | uniq -c
```

Screenshot 6: Pipes (|)



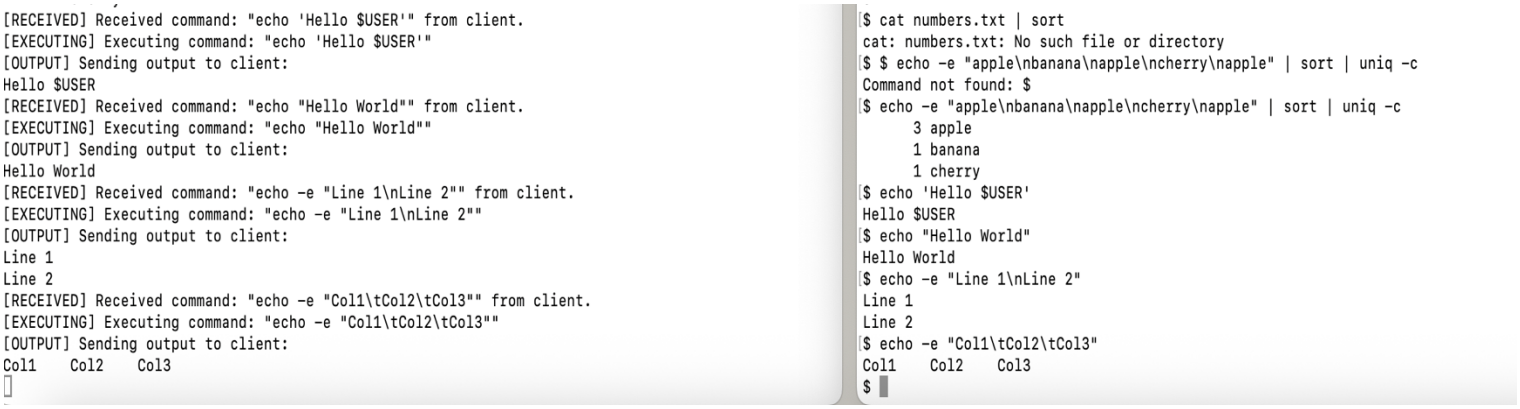
Explanation: Pipeline tests validate one of the most powerful shell features: connecting multiple commands where the output of one becomes the input of the next. The parser must split the command string on unquoted pipe characters, creating separate Command structures for each stage. The executor creates N-1 pipes for N commands, forks N child processes, and carefully manages file descriptors: each command's stdout connects to the next command's stdin via a pipe. All unused pipe file descriptors must be closed in each child to prevent resource leaks and ensure proper EOF propagation. The parent process closes all pipe descriptors and waits for all children to complete. Simple two-command pipes test basic functionality, while three-command pipelines test the executor's ability to coordinate multiple processes and multiple pipes simultaneously. These tests validate complex inter-process communication and demonstrate that data flows correctly through multiple processing stages without corruption or loss.

7. Quotes Handling

Test Command:

```
$ echo 'Hello $USER'
$ echo "Hello World"
$ echo -e "Line 1\nLine 2"
$ echo -e "Col1\tCol2\tCol3"
```

Screenshot 7: Quotes Handling



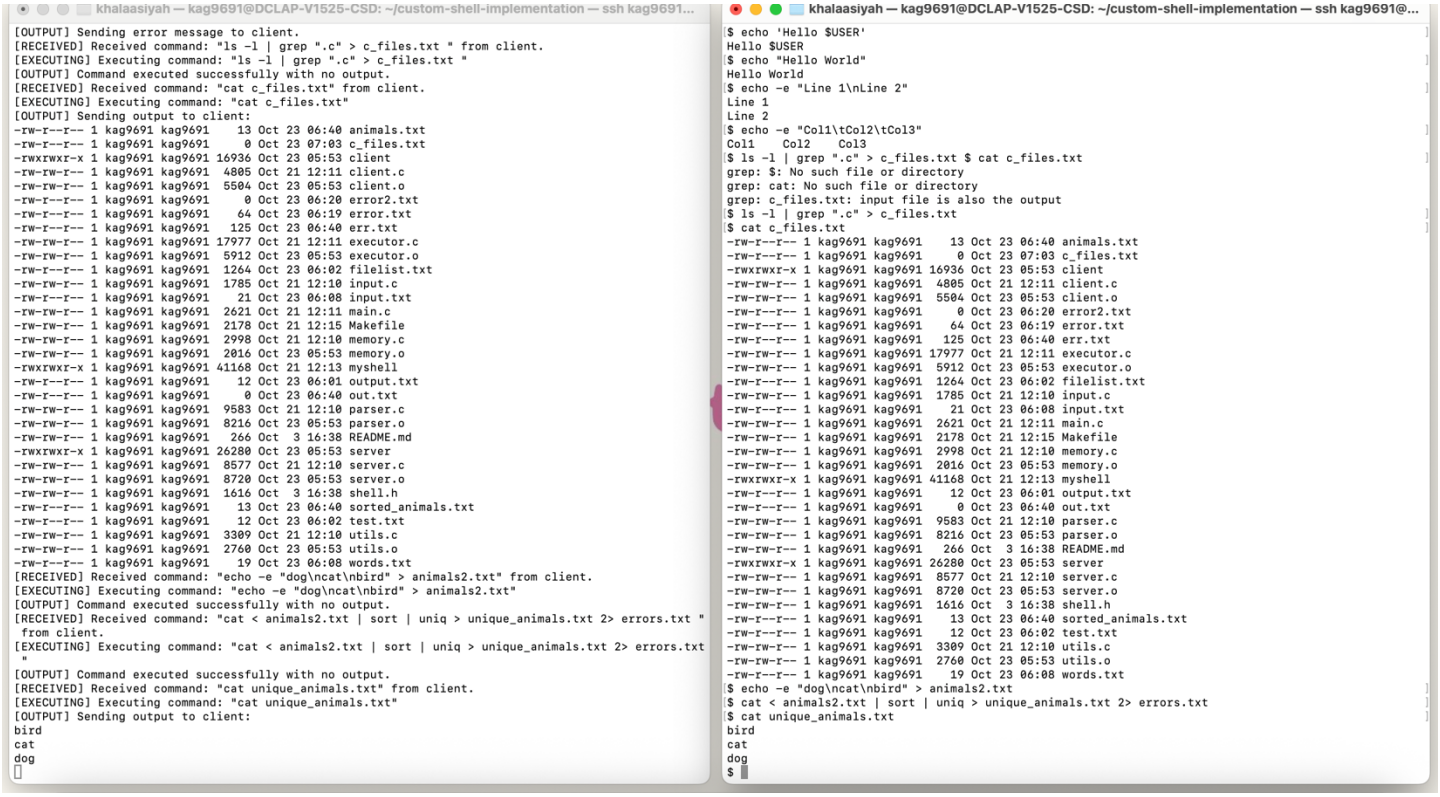
Explanation: Quote handling tests ensure the parser correctly processes single and double quotes according to shell conventions. Single quotes preserve everything literally—no variable expansion, no escape interpretation. Double quotes group words with spaces into a single argument while still allowing escape sequences. The tokenizer must track quote state (in_single_quote, in_double_quote) and remove the quote characters themselves while preserving the content. Quotes prevent the shell from interpreting special characters like spaces, pipes, and redirections as separators or operators. The test with apostrophe inside double quotes validates that quotes can be nested/mixed. The special characters test confirms that characters which normally have special meaning in shells (like !, $, *, &) are treated as literal text when quoted. These tests are essential because improper quote handling breaks commands with filenames containing spaces or special characters.

8. Complex Pipelines with Redirections

Test Command:

```
$ ls -l | grep ".c" > c_files.txt
$ cat c_files.txt
$ echo -e "dog\ncat\nbird" > animals2.txt
$ cat < animals2.txt | sort | uniq > unique_animals.txt 2> errors.txt
$ cat unique_animals.txt
```

Screenshot 8: Complex Pipelines with Redirections



Explanation: Complex scenario tests combine multiple shell features to validate real-world usage patterns. The first test combines pipelines with output redirection: data flows through multiple commands and the final output goes to a file instead of the terminal. The second test combines input redirection with pipelines: the first command reads from a file, and its output feeds the pipeline. The third test is the ultimate integration test, combining input redirection, multiple pipeline stages, output redirection, and error redirection all in one command. The executor must: (1) open the input file for the first command, (2) create pipes between commands, (3) open output file for the last command's stdout, (4) open error file for the last command's stderr, and (5) coordinate all file descriptor manipulations across multiple child processes. These tests prove the shell implementation is robust and can handle sophisticated command compositions that professional users expect. They validate that all features integrate seamlessly without conflicts or bugs.

Challenges

1. I/O Redirection for Capture vs. Execution

The primary challenge was adapting the existing execution logic, which assumes I/O is connected to a terminal, to a model where I/O must be captured and transmitted over a socket.

- **Resolution:** The `execute_pipeline` function from Phase 1 had to be wrapped by a new function, `capture_command_output` in `server.c`. This wrapper leverages **pipes** to create a non-blocking channel for the command's stdout and stderr, replacing the default terminal output file descriptors *only* for the command's child process using `dup2()`.

2. Differentiating Empty Output

A command like `mkdir new_dir` is successful but produces no standard output. Without a solution, a client would receive zero bytes and assume the connection was severed, leading to a confusing error message.

- **Resolution:** A dedicated **Empty Response Marker** (`\x01`) was introduced. The server now checks if the combined stdout/stderr is empty; if so, it sends the marker. The client's receive loop checks for a 1-byte response containing this marker and, if found, simply loops to the next prompt, providing a seamless user experience for successful non-verbose commands.

Division of Tasks

This project was completed individually. I was responsible for every part, including:

Server Implementation & Core Integration: Socket creation, binding, listening, and client handling loop in `server.c`. Implementation and debugging of the critical `capture_command_output` function using pipes and `fork/dup2`. Integration of the Phase 1 shell core objects (`parser.o`, `executor.o`, etc.) into the server executable.

Client Implementation: User input handling (`read_input`, `display_prompt`) and network connection/communication logic in `client.c`. Implementation of response handling, including the logic for the `EMPTY_RESPONSE_MARKER`. Writing the `Makefile` and extensive testing of the full client-server system.

References

1. Project Phase 2 Instructions.
2. Garrett, Khaleeqa Aasiyah. *Phase 1 Implementation Report*. Operating Systems Fall 2025.
3. Stevens, W. Richard, and Stephen A. Rago. *Advanced Programming in the UNIX Environment*. 3rd ed., Addison-Wesley, 2013.
4. Silberschatz, Abraham, Peter Baer Galvin, and Greg Gagne. *Operating System Concepts*. 10th ed., Wiley, 2018.
5. *Various Linux Manual Pages (man pages)* (e.g., `access(2)`, `dup2(2)`, `execvp(3)`, `fork(2)`, `pipe(2)`).